

📖 Week 1 — Practice Problems

Instructions: For each problem, write a Python script (.py file), run it, and verify your output matches the expected result. Try to solve each problem **without looking at the Solutions file** first. The learning happens in the struggle.

No solutions in this file. See [Solutions.md](#) after you have attempted each problem.

Problem 1 — Personalized Greeting

Objective: Practice using `print()` to produce formatted, structured output.

Write a Python script that prints the following output **exactly** (replace `[Your Name]` with your actual name):

```
=====
Welcome, [Your Name]!
I am learning Python.
Today is a great day to code.
=====
```

Requirements:

- Use only `print()` statements
- The `=====` lines must be at least 28 characters long
- Your name must appear on the second line
- The script must be saved as `greeting.py` and run from the terminal

Hints:

- You can create a line of `=` characters by typing them directly as a string inside `print()`
- You can also try `print("=" * 28)` — experiment and see what happens!
- There is a space before `Welcome` — look carefully at the expected output

Common Mistake to Avoid: Don't forget that `print()` adds a newline at the end of each call. Each line of output = one `print()` call.

Challenge Extension: Make the greeting dynamic — print the number of characters in your name on a line below. For example: `Your name has 4 characters.`

Problem 2 — Python vs. Compiled Languages

Objective: Practice writing multiple `print()` statements and using comments effectively.

Python is an **interpreted language**. Write a script that explains what this means. Your script must:

- Have a clear title printed at the top (e.g., Python: Interpreted vs. Compiled)
- Include **at least 3 `print()` statements** explaining the difference
- Have **at least 3 comments** (using `#`) explaining what each section of code does
- Print a separator line between sections

Example output structure (you write the content):

```
=== Python: Interpreted vs. Compiled ===

Interpreted languages run code line by line.
Compiled languages convert all code before running.
Python is interpreted, which makes development faster.
```

Hints:

- Think about: What happens when Python runs a script? How is that different from a language like C?
- You can have blank lines in output using `print()` with no arguments
- Your explanation doesn't need to be long — clear and accurate is better than lengthy

Common Mistake to Avoid: Comments are for **humans** (using `#`). Don't confuse them with text displayed by `print()`.

Challenge Extension: Add a third section to your script listing 3 pros and 3 cons of interpreted languages.

Problem 3 — Comments Practice

Objective: Practice writing meaningful comments and using `print()` with multiple data types.

Write a Python script with **at least 5 lines of code** where:

- Every line of code has a **comment above it** explaining what that line does
- You use `print()` to display at least **3 different data types**: a string, a number (integer or float), and a boolean
- The output is clean and readable — label each printed value

Requirements:

- Minimum 5 `print()` statements
- Minimum 5 comments (one per code line, above each statement)
- Must display: at least 1 string, 1 number, 1 boolean

Example of a well-commented script (structure only — write your own content):

```
# Display a welcome heading
print("--- My Python Practice Script ---")

# Print a text description
```

```
print("Language: Python")

# Print a whole number
print(2024)

# Print a decimal value
print(3.14159)

# Print a True/False value
print(True)
```

Hints:

- Write the comment first, then write the code line below it
- Make your comments describe **what** the line does, not just repeat what it says
 - Bad comment: `# prints True`
 - Good comment: `# Print whether the user is logged in (boolean value)`

Common Mistake to Avoid: Don't put the comment on the same line as code if you want it above — it becomes an inline comment, which is a different style.

Challenge Extension: Add a section that prints all three data types on a single line using commas, like: `String: Python | Number: 42 | Boolean: True`. Use `sep=""` or `end=""` to format it nicely.

III Problem Difficulty Guide

Problem	Difficulty	Key Skill
Problem 1	☆ Beginner	<code>print()</code> output formatting
Problem 2	☆☆ Easy	Multiple prints + comments for explanation
Problem 3	☆☆ Easy	Comments discipline + data type printing
Challenge Lab	☆☆☆ Intermediate	Creative use of <code>print()</code>

☐ Self-Check Before Checking Solutions

Before opening `Solutions.md`, answer these:

- ☐ Did your output match the expected format?
- ☐ Did you run the script from the terminal (not just "preview" it)?
- ☐ Did you use at least one comment in each script?
- ☐ Could you explain what each line of your code does out loud?

If you answered yes to all four — excellent. Check your solutions and note any differences.

☐ Cross References

Previous Topic	Lab 1.2 — Understanding print()
Next Topic	Solutions
Challenge	Challenge Lab — ASCII Art
Week README	Week 1 Overview

← [Lab 1.2](#) | Next: [Solutions](#) →